COMP 469
Fall 2026

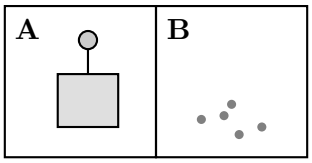
Discussion 2 Solutions

Intelligent Agents (AIMA 4th ed., Chapter 2)

Instructor notes. Budget 50–60 minutes. Suggested pacing: Q1 (8 min), Q2 (15 min), Q3 (15 min), Q4 (8 min), Q5 (12 min). Q2(c) and Q3(b) are the two parts worth reserving whiteboard time for; both reward argument over recall. Common misconceptions to watch for are flagged in the individual solutions below.

Q1. The Vacuum World, Formalized

Consider the vacuum world of Figure 2.2, generalized to a $1 \times n$ row of squares ($n \geq 2$). Each square is either *Clean* or *Dirty*. The agent occupies exactly one square at a time. Its sensors report the label of its current square and whether that square contains dirt. Its actions are $\{Left, Right, Suck, NoOp\}$. Sucking cleans the current square, clean squares stay clean, and a move that would leave the row leaves the agent where it is.



(a) State precisely the difference between an **agent function** and an **agent program**. Which of the two can depend on the entire percept history, and what must the other one do to obtain the same behavior?

Solution. The **agent function** is an abstract mathematical mapping from every possible *percept sequence* to an action. It is an external characterization of behavior; it does not say how the behavior is produced.

The **agent program** is a concrete implementation running on an architecture (*agent* = *architecture* + *program*). It takes only the *current percept* as input, because that is all the sensors deliver at that instant.

Only the agent function may depend on the whole history. If the desired behavior depends on the history, the program has to store what it needs in **internal state** across calls. That requirement is exactly what separates a simple reflex agent from a model-based one, and it is the crux of Q2(c).

(b) How many distinct **environment states** are there? Justify your count.

Solution. $n \cdot 2^n$. The state is (agent location, dirt configuration): n choices of square for the agent, and each of the n squares independently Clean or Dirty, giving 2^n dirt configurations. For the two-square world of Figure 2.2 this is $2 \cdot 2^2 = 8$ states.

Common error: answering 2^n (forgetting the agent's own position is part of the state) or $n + 2^n$ (adding instead of multiplying independent factors).

(c) How many distinct percepts can the agent receive? How many distinct percept sequences of length at most T are there? Give a closed form.

Solution. A percept is (location, dirt status of that square), so there are $|P| = 2n$ distinct percepts.

The number of percept sequences of length exactly t is $(2n)^t$, so the number of sequences of length at most T is

$$\sum_{t=1}^{T}(2n)^t \;=\; \frac{(2n)^{T+1} - 2n}{2n - 1}.$$

This is precisely the number of rows a TABLE-DRIVEN-AGENT would need (AIMA §2.4.1). For $n = 2$ and a modest lifetime of $T = 20$ that is already about 1.5×10^{12} rows.

(d) How many distinct agent functions are there over percept sequences of length at most T? Evaluate your expression for $n = 2$, $T = 2$, and say in one sentence what the number implies about building agents by table lookup.

Solution. Each of the $N = \sum_{t=1}^{T}(2n)^t$ sequences may be mapped independently to any of the $|A| = 4$ actions, so there are $|A|^N = 4^N$ agent functions.

For $n = 2$, $T = 2$: $N = 4^1 + 4^2 = 20$, so $4^{20} \approx 1.1 \times 10^{12}$ agent functions — for a two-square world with a two-step lifetime.

Implication: the space of behaviors is astronomically larger than the space of *sensible* behaviors, and tabulating it is hopeless. Even a correct table is (a) unstorable, (b) unwritable by a designer, and (c) unlearnable from experience. The design problem is to produce rational behavior from a small program, not from a vast table.

Q2. Rationality Is Relative

Work in the two-square world (squares A and B). Consider the agent whose agent function is: *if the current square is dirty then Suck; otherwise move to the other square.* Assume the geography is known a priori, but the initial dirt distribution and initial location are not; sensors are accurate; clean squares stay clean.

(a) Under performance measure M_1 — one point for each clean square at each time step over a lifetime of 1000 steps — is this agent rational? Justify against the four things rationality depends on.

> **Solution.** Yes. Rationality depends on (1) the performance measure, (2) the agent's prior knowledge of the environment, (3) the available actions, and (4) the percept sequence to date. Under M_1:
>
> - Dirt is only removable by *Suck*, and the agent sucks whenever it perceives dirt, so it never delays a cleaning it could have performed.
> - The only way to score higher would be to clean a square before ever perceiving it, which requires knowledge the agent does not and cannot have (it does not know the initial dirt distribution).
> - Movement is free under M_1, so the post-cleanup oscillation costs nothing.
>
> Its expected score is therefore at least as high as any other agent's, which is the definition we need. Note what this argument is *not*: it is not a claim that the agent is well designed in general, only that it is rational *relative to M_1 and these assumptions.*

(b) Now use M_2: identical to M_1, but with a penalty of one point for each movement action. Is the same agent still rational? Quantify the loss.

> **Solution.** No. Let t_0 be the step at which the last dirty square is cleaned. From t_0 onward there is nothing left to gain, but the agent still alternates *Left*/*Right* forever, paying 1 point per step. Against an agent that does *NoOp* after t_0, it loses approximately $1000 - t_0$ points — on a clean-ish start, nearly the full 1000.
>
> The agent function did not change; the yardstick did. This is the point of the question: rationality is a property of the *agent–environment–measure triple*, not of the agent alone.

(c) Show that *no* simple reflex agent is rational under M_2, and name the smallest capability that has to be added.

> **Solution.** A simple reflex agent maps the *current percept* to an action. In this world the percept $[A, \mathit{Clean}]$ is identical whether or not B has already been cleaned, so the agent must commit to a single response to it. Enumerate the options:
>
> - If it *moves* on *Clean*, then once both squares are clean it moves forever, paying the M_2 penalty forever — irrational.
> - If it does *NoOp* on *Clean*, then a start in a clean A with dirt in B means it never crosses to B, and B is never cleaned — also irrational.
> - Randomizing between these two only mixes the two failures; the movement penalty is

still paid indefinitely with positive probability per step.

So no mapping from the current percept alone is rational under M_2.

The missing capability is **internal state**: memory that both squares have been observed clean. That upgrades the design to a **model-based reflex agent**, which needs a transition model ("sucking cleans the current square; clean squares stay clean; *Left*/*Right* change my location") and a sensor model. With that state, the rational policy is: clean what you see, sweep once to the other square, then *NoOp* forever.

(d) Suppose clean squares can spontaneously become dirty again. Which environment properties change, and why is "*NoOp* forever" no longer rational?

Solution. The environment becomes **nondeterministic** (the next state is no longer fixed by the current state and action) and **dynamic** (it changes while the agent deliberates or idles). It remains partially observable to an agent with only a local dirt sensor: a square the agent is not standing on may have become dirty without the agent knowing.

"*NoOp* forever" is now irrational because dirt accumulates unboundedly, costing a point per dirty square per step, while patrolling costs only the movement penalty. The rational policy has to trade the movement penalty against the expected rate of dirt appearance — an expected-utility calculation rather than a goal test, which is why this version of the problem belongs to Chapters 16–17 rather than Chapter 2.

(e) Remove the location sensor: percepts are now only [*Dirty*] or [*Clean*]. Argue that no deterministic simple reflex agent cleans both squares from every start state, and show that a randomized one does. If the agent flips a fair coin between *Left* and *Right* on [*Clean*], what is the expected number of actions to reach the other square?

Solution. On [*Dirty*] the only sensible action is *Suck*. The failure is in the response to [*Clean*], which a deterministic agent must fix once and for all:

- Always *Left*: if the agent starts in a clean A with dirt in B, *Left* bumps the wall and it stays in A forever. B is never cleaned.
- Always *Right*: symmetric failure starting from a clean B with dirt in A.
- Always *NoOp*: fails in both of the above.

So each deterministic choice has a start state on which it loops or stalls forever — the classic infinite loop of a simple reflex agent in a partially observable environment.

With a fair coin, each action independently moves the agent to the other square with probability $1/2$ (the other outcome bumps the wall). The number of actions until the first success is geometric with $p = 1/2$, so

$$\mathbb{E}[\text{actions}] = \frac{1}{p} = 2.$$

The agent therefore reaches the dirty square in two moves on average and cleans it, in finite expected time from every start state. A randomized simple reflex agent strictly outperforms every deterministic one here — a useful trick under partial observability,

though in single-agent environments randomization is usually a symptom that a model-based design would do better.

Q3. PEAS and the Seven Dimensions

A university deploys an agent on its inbound mail gateway. For each arriving message the agent chooses one action: *deliver*, *deliver with a warning banner*, *quarantine*, or *escalate to a human analyst*. Attackers adapt their campaigns in response to what gets through.

(a) Give a PEAS description for this task environment. Be specific; "be accurate" is not a performance measure.

Solution. One reasonable answer (students should produce concrete, measurable items in P, and should not collapse Environment into Sensors):

- **Performance:** fraction of malicious mail quarantined; false-positive rate on legitimate mail (weighted higher — a quarantined payroll notice is costly); time-to-verdict; analyst hours consumed by escalations; expected loss from a missed credential-harvesting message.
- **Environment:** the inbound message stream; mailboxes and the users reading them; sending infrastructure and its reputation history; the attackers themselves; the quarantine and release workflow; the analysts.
- **Actuators:** deliver, banner, quarantine, escalate; also release-on-appeal, URL rewriting, attachment stripping, and (if in scope) a gateway block on the sender.
- **Sensors:** headers and authentication results (SPF/DKIM/DMARC), body text, attachments and detonation results, URL reputation and sandbox verdicts, sender history and volume, and user-submitted reports.

Grading note: award credit for a performance measure written in terms of *outcomes in the environment* (mail correctly handled, loss avoided) rather than in terms of agent behavior ("flags suspicious mail"). That distinction is the setup for Q5(e).

(b) Classify this environment along all seven dimensions and justify the two you find most debatable.

Solution.

- **Partially observable:** the sender's intent, and whether a URL will be weaponized after delivery, are not in the percept.
- **Multiagent, and competitive:** attackers change behavior in response to what the filter does. This is the textbook test — B's behavior is best described as maximizing a measure that depends on A's behavior.
- **Nondeterministic:** the same verdict on the same message does not determine what the user or the attacker does next.
- **Sequential (debatable):** treating each message as an isolated classification makes it look episodic, and per-message it nearly is. But a quarantine decision feeds retraining data, a gateway block changes the attacker's next move, and a campaign spans many messages — so the current decision affects future decisions. Accept "episodic" only with an argument that scopes the agent to a single stateless classification.
- **Dynamic:** mail keeps arriving and linked content can change while the agent delib-

erates. "Semidynamic" is defensible if the only thing degrading during deliberation is the score (delivery latency).

- **Discrete:** messages and actions are discrete, though many features are real-valued.
- **Partly unknown:** the rules of email are known; the attacker's current tooling and techniques are not, which is why the agent must learn.

(c) Fill in the table. Use the vocabulary from AIMA Figure 2.6.

Task environment	Observable	Agents	Determ.	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Determ.	Sequential	Static	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Warehouse pallet robot	Partially	Multi	Nondeterm.	Sequential	Dynamic	Continuous
Mail-gateway agent (above)	Partially	Multi	Nondeterm.	Sequential	Dynamic	Discrete

Solution. Points worth drawing out:

- Crosswords are *sequential*, not episodic — an early wrong entry constrains every later one — and *static*, since the puzzle does not change while you think.
- Poker is *stochastic* rather than merely nondeterministic: the model quantifies the probabilities of the deal, and the agent uses them. Reserve "nondeterministic" for models that list outcomes without probabilities.
- The warehouse robot is partially observable because occlusion and sensor noise hide parts of the floor; it is multiagent because other robots and humans share the space; it is continuous in state, time, and action (pose, velocity, steering).
- Nothing in this table is a "known/unknown" column — see part (d).

(d) Why is **known vs. unknown** not a property of the environment at all? Give one environment that is known but partially observable, and one that is unknown but fully observable.

Solution. Known/unknown describes the *agent's or designer's* state of knowledge about the environment's transition model — what the actions do — rather than any physical feature of the environment. Two environments identical in every physical respect can be known to one agent and unknown to another.

- **Known but partially observable:** solitaire. You know the rules completely, but you cannot see the face-down cards.
- **Unknown but fully observable:** a new video game. The screen shows the entire game state, but you do not yet know what the buttons do.

Q4. Choosing an Agent Architecture

For each scenario, select the **simplest** agent architecture that suffices. Assume each scenario is independent.

(a) A thermostat switches the heater on whenever the measured temperature falls below the set-point and off when it rises above.

● simple reflex ○ model-based reflex ○ goal-based
○ utility-based

Solution. Simple reflex. The condition–action rule *if temp-below-setpoint then heat-on* reads everything it needs from the current percept; no history, no goal representation, no tradeoff.

(b) A camera-based braking agent must decide whether the car ahead is braking. On older vehicles, taillights and brake lights are not distinguishable in a single frame — only the transition from off to on identifies braking.

○ simple reflex ● model-based reflex ○ goal-based
○ utility-based

Solution. Model-based reflex. The condition "car in front is braking" is not determined by the current percept; the agent must retain the previous frame. That is internal state, plus a sensor model relating illuminated red regions to the world.

(c) A campus delivery robot operates in a fully mapped building. Its destination changes from run to run, and it must plan a route to whichever room it is given.

○ simple reflex ○ model-based reflex ● goal-based
○ utility-based

Solution. Goal-based. The right action depends on where the robot is trying to get to, so the goal must be represented explicitly and combined with the model to search for an action sequence. A reflex rule set would have to be replaced wholesale for each new destination — the flexibility argument of AIMA §2.4.4.

(d) The same robot must now weigh delivery speed against battery consumption and the risk of collisions in crowded corridors, and no route satisfies all three.

○ simple reflex ○ model-based reflex ○ goal-based
● utility-based

Solution. Utility-based. Goals give only a binary happy/unhappy split. With conflicting objectives that cannot all be met, the agent needs a utility function to specify the tradeoff, and under uncertainty it should maximize *expected* utility.

(e) A vacuum agent with location and dirt sensors must stop moving once the entire floor is clean, under a performance measure that penalizes movement.

○ simple reflex ● model-based reflex ○ goal-based
○ utility-based

Solution. Model-based reflex, by the argument of Q2(c). Students who answer "goal-based" should be pushed on it: the agent needs no search over action sequences and no explicit goal representation — remembering that both squares were seen clean is enough.

Mark each statement **T** or **F** and give a one-line reason.

A simple reflex agent can be rational.	T
Rationality requires knowing the actual outcome of one's actions.	F
Every utility-based agent must also be a model-based agent.	F
An autonomous agent is one that ignores knowledge supplied by its designer.	F
In a fully observable, deterministic environment, an agent need not maintain internal state to keep track of the world.	T
Randomized behavior is never rational.	F

Solution.

1. **T.** The vacuum agent of Q2(a) is a simple reflex agent and is rational under M_1. Simplicity of the program says nothing about rationality; the measure and environment do.
2. **F.** That is *omniscience*, which is impossible. Rationality maximizes *expected* performance given the percept sequence to date; perfection maximizes actual performance. Crossing the street and being hit by falling cargo does not make the crossing irrational.
3. **F.** Model-free agents can learn which action is best in a situation without ever learning how the action changes the world (Chapters 22 and 26).
4. **F.** Autonomy means learning to compensate for partial or incorrect prior knowledge — not discarding prior knowledge. Complete autonomy from the start is unreasonable; an agent with no experience and no built-in knowledge can only act randomly.
5. **T.** With full observability and determinism there are no unobserved aspects to track, so no internal world model is needed. (The agent may still keep state for other reasons, such

as holding a plan.)

6. **F.** Randomization can be rational in competitive multiagent environments, where predictability is exploitable, and it rescues simple reflex agents from infinite loops under partial observability, as in Q2(e).

Q5. Learning Agents, Representations, and Bad Yardsticks

(a) A learning agent has four conceptual components. Name the component responsible for each job below.

Selects the external action to take right now.	Performance element
Tells the agent how well it is doing against a fixed performance standard.	Critic
Modifies the agent's components so that it does better in future.	Learning element
Suggests deliberately suboptimal actions that would produce informative experience.	Problem generator

Solution. The performance element is what we called "the whole agent" in §2.4.2–2.4.5: percepts in, actions out. The learning element makes the improvements, the critic supplies the judgment it needs, and the problem generator drives exploration — what a scientist does when running an experiment.

(b) Why must the performance standard be *fixed* and conceptually *outside* the agent? What goes wrong otherwise?

Solution. Because the learning element modifies the agent's components, and if the standard were one of those components the cheapest "improvement" available would be to lower the bar rather than raise performance. An agent that can rewrite its own yardstick will optimize the yardstick.

Note also why the critic is needed at all: percepts do not announce success. A chess program can perceive that it has checkmated its opponent, but nothing in that percept says checkmate is good; the fixed standard supplies that.

(c) Classify each representation as **atomic**, **factored**, or **structured**.

The state is the name of the city the driver is currently in.	Atomic
The state is a vector: GPS coordinates, fuel level, toll money, oil-light status.	Factored
"A truck ahead is reversing into a driveway that a loose cow is blocking."	Structured
The state is one of N hidden labels in a hidden Markov model.	Atomic
A complete assignment to the variables of a constraint satisfaction problem.	Factored

Solution. The axis runs from atomic (states are indivisible black boxes; only equality is observable — search, HMMs, MDPs) through factored (a fixed set of variables with values, so two states can share some attributes and differ in others — CSPs, propositional logic, Bayes nets) to structured (objects and their relations, described explicitly — first-order logic, relational databases, much of language).

The reason it matters: expressiveness buys conciseness — the rules of chess take a page or two in first-order logic, thousands of pages in propositional logic, and roughly 10^{38} pages as a finite-state automaton — but reasoning and learning get harder as expressiveness grows. Real systems often operate at several points on the axis at once.

(d) Explain why a **distributed** representation degrades more gracefully under bit corruption than a **localist** one.

Solution. In a localist representation the mapping from concept to memory location is arbitrary, so corrupting a few bits can land on an unrelated concept — *Truck* becomes *Truce*. In a distributed representation a concept is a point in a high-dimensional space, spread over many locations, and each location participates in many concepts; corrupting a few bits moves you to a nearby point, whose meaning is similar. The error degrades meaning smoothly instead of substituting nonsense.

(e) A vendor proposes to evaluate a cleaning agent by *the amount of dirt collected in an eight-hour shift.*

(i) Describe the degenerate policy a rational agent would adopt under this measure.

(ii) Propose a measure that does not admit that exploit.

(iii) State the general design rule this illustrates, and say what an agent designer should do when the true measure cannot be written down correctly in advance.

Solution.

(i) Clean up the dirt, dump it back on the floor, clean it up again, and repeat. Total dirt

collected is maximized; the floor is no cleaner at the end of the shift than at the start. With a rational agent, what you ask for is what you get.

(ii) Score the *state of the environment*: one point per clean square per time step, with penalties for electricity and noise. There is no way to inflate this by manufacturing work, because the score never rewards the collecting, only the being-clean.

(iii) The rule: design performance measures according to what you actually want achieved *in the environment*, not according to how you imagine the agent should behave. Measures written over agent behavior invite the agent to perform the behavior instead of the outcome.

When the true measure cannot be specified correctly in advance — because the designer is unsure how to write it, or because different users have different preferences — the right response is not to guess and freeze it. It is to design agents that hold *uncertainty about the true objective* and learn more about it from interaction with the user, which also gives them a reason not to resist correction. This is the King Midas problem of §1.5, taken up in Chapters 16, 18, and 22.

Discussion prompt if time allows. Ask the group for a measure from their own experience with the same defect — tickets closed per hour, alerts triaged per shift, lines of code written. Every one of them is a measure over agent behavior rather than environment outcomes, and every one has a well-known degenerate policy.